Facultad de Ingeniería

Optimización

TRABAJO GRUPAL

MODELADO, IMPLEMENTACIÓN Y ANÁLISIS DE UN PROBLEMA NP-COMPLETO

Autores:

Felipe Sanhueza

Benjamín Gallegos

Felipe Levi

Profesor:

Luis Fuentes Larenas

Santiago, Chile

2026

1. DEFINICIÓN DEL PROBLEMA

a) <u>Descripción del problema.</u>

- El **problema de Programación Entera 0-1** consiste en seleccionar el mejor subconjunto de opciones a partir de un grupo finito de alternativas que son indivisibles. El objetivo en este caso es **maximizar un beneficio global,** o bien, **minimizar un costo,** sujeto a un inventario limitado de recursos. La característica fundamental de este problema es que las decisiones son estrictamente binarias o dicotómicas: una opción se acepta por completo o se rechaza de forma definitiva.

b) <u>Definición formal.</u>

- Desde una perspectiva algebraica, la versión de optimización del problema se define de la siguiente manera:

  Entrada:

- Un vector de beneficios/costos c
- Una matriz de coeficientes tecnológicos A
- Un vector de capacidades b

  Salida:

- Un vector de decisión discreto $x \in \{0, 1\}^n$ que cumple dos condiciones de forma simultánea:
  1. Maximiza la función objetivo: $max\ c^t x$
  2. Satisface el sistema de restricciones de recursos: $Ax \leq b$

c) <u>Ejemplo Ilustrativo: Asignación Estratégica de Capital.</u>

- **Contexto:** Una empresa tiene como presupuesto máximo de $14.000 para invertir en cuatros posibles proyectos indivisibles.

1. **Proyecto 1:** Costo $5.000 | Beneficio (VPN) $8.000
2. **Proyecto 2:** Costo $7.000 | Beneficio (VPN) $11.000
3. **Proyecto 3:** Costo $4.000 | Beneficio (VPN) $6.000
4. **Proyecto 4:** Costo $3.000 | Beneficio (VPN) $4.000

- Para modelar esto, definimos primero la variable $x_j \in \{0, 1\}$, donde $x_j = 1$ si se invierte en el Proyecto j, y 0 en caso contrario.
- Aquí el objetivo es maximizar el VPN total, considerando la restricción de no superar las $14.000. Sin embargo, para este caso, es necesario enfocarnos en la naturaleza del problema, y en dos tipos de soluciones.
- **Solución fraccionaria (Irreal):** Si el problema no fuera binario, la matemática sugeriría invertir en el Proyecto 1, el Proyecto 2 y usar el resto del dinero para financiar el 50% del Proyecto 3. Esto daría en teoría un beneficio de $22.000.
- **Solución 0-1 (Real):** Como los proyectos no se pueden fraccionar, la verdadera solución óptima es ejecutar solo los Proyectos 1 y 2. Esta elección consume $12.000 del presupuesto, y entrega el máximo beneficio real factible, de $19.000.

d) <u>Aplicaciones en el mundo real.</u>

Las variables binarias son vitales para poder modelar decisiones lógicas y costos fijos en las distintas industrias. A continuación, se indican algunas de estas aplicaciones:

- **Asignación de Horarios:** Creación de turnos para personal médico, o bien tripulaciones aéreas, donde la variable 0-1 decide si un trabajador cubre algún bloque en específico.
- **Ubicación de Instalaciones:** Decisión de abrir o mantener cerrado un centro de distribución, en donde se asume un costo fijo de apertura. Lo anterior tiene como finalidad maximizar el nivel de servicio al cliente y satisfacer las demandas.
- **Enrutamiento y redes:** Optimización de trayectorias, como el **problema del Vendedor Viajero**, donde las variables indican si un vehículo transita o no por una ruta específica entre dos ciudades.
- **Biología Computacional:** Modelado de secuencias de ADN y mutaciones discretas para el diseño de tratamientos médicos.

2. MODELADO MATEMÁTICO

- Para resolver el problema de programación entera 0-1 mediante algoritmos, es necesario realizar la definición de ciertos componentes.

a) Conjuntos y Parámetros

- N: Conjunto de todas las opciones o decisiones posibles (**n** elementos)
- M: Conjunto de todas las restricciones o recursos limitados (**m** restricciones)
- c_j: Impacto (costo/beneficio) de activar la opción *j*.
- a_{ij}: Coeficiente tecnológico. Representa la cantidad del recurso *i* que consume la opción *j*.
- b_i: Límite máximo disponible del recurso *i*.

b) Variables de Decisión

El núcleo del modelo son sus variables, las cuales actúan de forma restrictiva como interruptores lógicos.

$$x_j \in \{0, 1\} \qquad \forall j \in N$$

- La definición anterior indica que x_j actúa puramente como una función indicadora, o bien un operador lógico booleano. Esto significa que cuando se observa un 1, quiere decir que la opción *j* es seleccionada, por lo que un 0 indica que la opción es rechazada.

c) Función Objetivo y Restricciones

Lo que busca el modelo es optimizar el objetivo global, respetando a la vez las capacidades implicadas. Por lo anterior, así se pueden determinar la función objetivo y las restricciones:

- **Función Objetivo Clásica:** $max \sum_{j=1}^{n} c_j x_j$

- **Restricciones de Capacidad base:** $\sum_{j=1}^{n} a_{ij} x_j \leq b_i \quad \forall i \in M$

- Además de las limitaciones de recursos físicos, las inecuaciones lineales permiten modelar el álgebra booleana para capturar relaciones complejas en el mundo real. Para este problema, se implementa el uso de inecuaciones dentro de las restricciones:

- **Condicionales:** Si la decisión A depende de que se apruebe B, se modela como $x_a \leq x_b$.

- **Mutuamente excluyentes:** Si solo se puede escoger una opción de un grupo *K*, se impone $\sum_{k \in K} x_k \leq 1$.

- **Lógica "Big M":** Permite vincular variables continuas (y) con decisiones binarias (x). Por ejemplo, la capacidad de transmisión y_i debe ser cero, si no se construye la antena ($x_j = 0$). La restricción se vería de la siguiente forma:
  $y_j \leq M \cdot x_j$ (donde *M* es un número muy grande)

d) Optimización vs. Versión de Decisión

Para comprender el problema 0-1 IP, es necesario hacer una distinción formal entre cómo se resuelve en la práctica operativa y cómo se analiza su dificultad teórica:

- Versión de Optimización: Corresponde a la Función Objetivo Clásica definida anteriormente. Su objetivo es encontrar el valor de decisión exacto que entregue el máximo/mínimo beneficio absoluto posible en el espacio de soluciones.
- Versión de Decisión: Para que la teoría de complejidad computacional pueda clasificar y medir la "dureza" del problema, se aísla la función objetivo y se transforma en una pregunta de evaluación logística, teniendo las opciones de Sí y No. Para ello, se establece un umbral de carácter escalar, denotado como *K*.

  Si tenemos este enfoque, ahora la formulación no busca el óptimo. En lugar de eso, se plantea la siguiente pregunta:

  ¿Existe al menos un vector binario $x \in \{0, 1\}^n$ que respete las restricciones ($Ax \leq b$) y logre una utilidad igual o superior a K ($c^T x \geq K$)?

3. ANÁLISIS DE COMPLEJIDAD COMPUTACIONAL Y CONTEXTO

- La dificultad del 0-1 IP no radica en formular las ecuaciones, sino en la escalabilidad exponencial al intentar resolverlo.

- Un problema determinado pertenece a la clase NP si demostramos verificar la validez de una solución propuesta en un tiempo polinomial. Si tenemos un vector x^t, debemos validarlo en tres pasos:

    1. **Tipado Binario:** Confirmar que $x' \in \{0, 1\}^n$ toma tiempo $O(n)$.
    2. **Cumplimiento de restricciones:** Multiplicar y verificar la matriz $Ax' \leq b$ toma tiempo $O(m\,x\,n)$.
    3. **Cumplimiento del umbral:** Calcular $c^T x' \geq K$ toma tiempo $O(n)$.

- Debido a que el costo computacional total para verificar la solución es $\approx O(m\,x\,n)$, queda demostrado que 0-1 IP pertenece a la clase NP. La función que representa el costo es de carácter polinómica, dependiente del tamaño de la entrada.
- Ahora bien, para que un problema sea considerado NP-completo, se deben cumplir dos condiciones:

    1. Estar en NP.
    2. Ser NP-arduo (o NP-hard): Esto significa que cualquier problema de la clase NP puede reducirse a 0-1 IP en tiempo polinomial.

- Para demostrar que el problema 0-1 IP es NP-arduo, se hace el uso de la técnica de Reducción Polinomial de Karp. El teorema demuestra que cualquier instancia del problema de Satisfacibilidad Booleana (3-SAT) puede transformarse en un modelo de 0-1 IP en tiempo polinomial. Es necesario seguir ciertas reglas para esta transformación:

1. <u>Variables Booleanas:</u> Cada proposición lógica u_j se reemplaza por una variable de decisión binaria $x_j \in \{0, 1\}$, donde 1 equivale a Verdadero y 0 a Falso.
2. <u>Negaciones Lógicas:</u> Si una fórmula requiere el literal positivo u_j, se escribe como x_j. Si requiere el literal negado $\neg u_j$, se traduce usando simetría aritmética como $(1 - x_j)$.
3. <u>Disyunciones:</u> En 3-SAT, cada cláusula exige que al menos un literal sea verdadero. Lo anterior provoca que la suma lineal de las variables que componen la cláusula sea mayor o igual a la unidad (≥ 1).

- A continuación se explica un ejemplo de transducción, dada una cláusula arbitraria de 3-SAT:

$$C_i = (u_1 \vee \neg u_4 \vee \neg u_7)$$

- Aplicando las reglas anteriores, se obtiene la siguiente restricción geométrica:

$$x_1 + (1 - x_4) + (1 - x_7) \geq 1$$

- Ahora, si agrupamos los términos y ordenamos la expresión, esto resulta en:

$$x_1 - x_4 - x_7 \geq -1$$

- Finalmente, multiplicamos por -1 para cumplir con el formato canónico estándar del modelo (de la forma $Ax \leq b$).

$$-x_1 + x_4 + x_7 \leq 1$$

- Al iterar este proceso sobre m cláusulas de una fórmula 3-SAT, se construye sistemáticamente la matriz de restricciones A y el vector límite b en un tiempo polinomial $O(m\,x\,n)$. De esta forma, si el sistema de inecuaciones $Ax \leq b$ tiene un

vector solución válido ($x \in \{0, 1\}^n$), significa que la fórmula lógica 3-SAT original es satisfacible. Esta transformación consolida la NP-completitud del problema de Programación Entera 0-1.

4. IMPLEMENTACIÓN MULTILENGUAJE

- Para llevar el problema 0-1 IP a código, hacemos uso del Problema de la Mochila 0-1 (Knapsack Problem) como caso de estudio. Teniendo en cuenta el tipo de problema, hay que destacar dos enfoques algorítmicos, en el cual uno tiene una solución base, y otro una solución mejorada, ambos programados nativamente en tres lenguajes distintos.

- **Solución Base: Búsqueda Exhaustiva.**
  Corresponde a un algoritmo puro de enumeración. Se construye un árbol de decisión binario donde se explora de forma recursiva la inclusión ($x_j = 1$) o la exclusión ($x_j = 0$) de cada variable. Solo se corta una rama si la suma de recursos consumidos supera la capacidad límite.

- **Solución Mejorada: Branch and Bound.**
  Esta solución añade heurísticas predictivas para descartar ramas enteras de forma definitiva. El proceso incluye un pre-ordenamiento por eficiencia (beneficio/costo), el cálculo de una cota superior en cada nodo, y una poda dinámica que elimina ramas si su cota superior es menor al mejor beneficio ya encontrado previamente.

- Con las soluciones ya definidas, ahora podemos detallar la arquitectura computacional específica y funciones utilizadas en cada lenguaje, y para cada tipo de solución.

a) IMPLEMENTACIÓN EN PYTHON

- El entorno de Python muestra una sintaxis muy flexible para implementar y auditar la simulación recursiva del árbol de búsqueda. Su estructura predeterminada basada en la herencia de referencias permite efectuar un backtracking de contexto directo,

aunque también hay que considerar su elevada latencia debido a su naturaleza de lenguaje interpretado.

- **Solución Base:** Se implementó mediante la clase *IPBase*, utilizando un método recursivo puro (*backtracking*). En lugar de manipular listas pesadas de estados, el algoritmo optimiza la memoria pasando el peso y beneficio acumulado (*peso_actual, beneficio_actual*) directamente como parámetros en la pila de llamadas. La recursión se bifurca limpiamente explorando la inclusión factible y la exclusión de cada ítem iterado.
- **Solución Mejorada:** Estructurada en la clase *IPMejorada.* El pre-ordenamiento heurístico se logra fusionando pesos y beneficios con *zip()* y ordenandolos descendentemente mediante una función *lambda* basada en su densidad. La cota superior se proyecta en el método *cota*, el cual aplica la relajación fraccional iterando sobre los elementos restantes. El algoritmo poda drásticamente la búsqueda cortando la ejecución si la cota fraccional no logra superar a la variable *mejor_beneficio.*

b) IMPLEMENTACIÓN EN C++

- El despliegue del algoritmo en C++ otorga un control nativo y determinista del hardware, permitiendo la máxima velocidad de ejecución y un monitoreo estricto de las iteraciones.

- **Solución Base:** El algoritmo se estructuró dentro de una clase (*SolverIPBase*), inicializando los vectores de datos (*const vector<double>&*). Esta arquitectura elude cualquier copia de memoria redundante durante las profundas llamadas recursivas. Adicionalmente, se incluyó un contador asintótico explícito (*nodos_visitados*) para registrar empíricamente la explosión exponencial de vértices evaluados.
- **Solución Mejorada:** La topología de ramificación se compacta definiendo un *struct Elemento* que encapsula peso, beneficio y ratio de cada opción. El ordenamiento inicial se delega a la función *std::sort* empleando un comparador personalizado (*comp*). Al igual que en la base, la clase *SolverIPMejorado* contabiliza los *nodos_visitados,* lo que permite ontrastar la abismal diferencia de esfuerzo computacional entre evaluar el hipercubo completo versus el espacio podado mediante la función (*cota*).

c) IMPLEMENTACIÓN EN JAVA

- A diferencia de los otros lenguajes, Java ofrece un entorno de Máquina Virtual (JVM) que requiere un diseño cuidadoso para evitar latencias estructurales durante la alta concurrencia recursiva.
- **Solución Base:** La ejecución estopa implementada en la clase *ip_base.* Para prevenir la sobresaturación del Recolector de Basura (Garbage Collector sweeps) generada por la instanciación repetitiva de objetos en cada ciclo, los parámetros físicos del problema se almacenaron estrictamente en arreglos primitivos globales (*double[] w, p*). Al igual que en C++, aquí se integra el atributo *nodos_Visitados* para la auditoría de rendimiento.
- **Solución Mejorada:** La heurística predictiva exigió un diseño orientado a objetos de forma estricta en la clase *ip_mejorada.* Se definió una clase interna estática (*static class Item*) que implementa la interfaz *Comparable<Item>*. Al sobreescribir el método *compareTo(),* se garantiza un ordenamiento descendente (type-safe) delegando la lógica a *Arrays.sort().* La poda temprana se ejecuta validando *cota()* contra el *mejorP* global, logrando esquivar la recursión inútil.

5. ANÁLISIS EXPERIMENTAL

Para evaluar el rendimiento real de los algoritmos y caracterizar su comportamiento predictivo, se diseñó un cuasi-experimento confrontando las implementaciones con instancias de volúmenes de coeficientes crecientes (desde N = 10 hasta N = 1000).

A continuación, se presentan las tablas maestras que consolidan el esfuerzo computacional medido en nodos visitados (Tabla 1), y los tiempos de resolución

(Tabla 2). Aquellas instancias donde el procesamiento excedió límites prácticos se han catalogado como "Inabordables".

N Evaluado	Ramas Teóricas Aprox. (2N)	Nodos Base (C++)	Nodos Base (Java)	Nodos Mejorada (C++)	Nodos Mejorada (Java)	Eficiencia de Poda
10	1,024	1,176	1,154	-	-	-
15	32,768	39,212	39,19	-	-	-
20	1,048,576	-	1,220,917	93	239	> 99.99 %
25	33,554,432	38,085,072	38,253,667	-	-	-
30	1,073,741,824	1,162,177,087	1,204,984,676	-	-	-
40	$\approx 1.09 \times 10^{12}$	Inabordable	Inabordable	134	337	> 99.99 %
60	$\approx 1.15 \times 10^{18}$	Inabordable	Inabordable	388	147	> 99.99 %
100	$\approx 1.26 \times 10^{30}$	Inabordable	Inabordable	590	1,614	> 99.99 %
1000	$\approx 1.07 \times 10^{301}$	Inabordable	Inabordable	16,108	8,31	> 99.99 %

Tabla 1: Carga computacional (nodos evaluados) y eficiencia heurística del algoritmo Branch and Bound frente al tamaño del hipercubo.

Lenguaje	Versión	N=10	N=15	N=20	N=25	N=30	N=40	N=60	N=100	N=1000
Python	Base	0.000143	0.004425	0.135019	4.556.120	140.394.441	Inabordable	Inabordable	Inabordable	Inabordable
Python	Mejorada	-	-	0.000045	-	-	0.000093	0.000251	0.000857	-
Java	Base	0.000163	0.000595	0.007195	0.109581	3.385.958	Inabordable	Inabordable	Inabordable	Inabordable
Java	Mejorada	-	-	0.000088	-	-	0.000198	0.000041	0.000119	0.003636
C++	Base	0.000010	0.000404	-	0.293404	10.733.893	Inabordable	Inabordable	Inabordable	Inabordable
C++	Mejorada	-	-	0.000005	-	-	0.000013	0.000026	0.000045	0.003313

Tabla 2: Tiempos de ejecución en segundos según tamaño del problema (n), enfoque y lenguaje.

- **<u>Tiempo de Ejecución y Escalabilidad</u>**

  Los datos empíricos tabulados demuestran la naturaleza NP-completa del problema 0-1 IP. Analizando la escalabilidad del enfoque de Fuerza Bruta, se observa un quiebre de rendimiento abrupto: mientras en $n = 15$ los tiempos son despreciables, y justo en $n = 30$ ocurre una explosión geométrica. A partir de este punto, cualquier instancia de $n \geq 40$ se vuelve matemáticamente inabordable en tiempos razonables mediante este método exploratorio ciego.

- **<u>Diferencias entre enfoques (Fuerza Bruta vs. Branch and Bound)</u>**

  Al cruzar los datos de tiempo con la métrica estricta de "Nodos Visitados" (Tabla 1), el impacto arquitectónico entre ambos enfoques queda expuesto con total claridad empírica:

  Enfoque Base (Backtracking): Al carecer de inteligencia predictiva, se ve obligado a descender por casi todas las ramas del árbol binario. Para un problema de $n = 30$, los lenguajes evaluaron más de 1.160 millones de nodos, acercándose al límite teórico de combinaciones posibles.

  Enfoque Mejorado (Branch and Bound): La inclusión de la poda heurística esteriliza por completo la maldición exponencial. Al ordenar las variables por densidad e inyectar una cota superior fraccional, el algoritmo logró una

eficiencia de poda superior al 99.99%. Para la instancia de $n = 1000$ (con un hipercubo teórico de $\approx 1.07 \times 10^{301}$ ramas latentes), el modelo en C++ encontró el óptimo visitando únicamente 16,108 nodos, transformando un escenario inabordable en una resolución prácticamente instantánea.

- **<u>Diferencias entre lenguajes de programación</u>**

  El experimento estandarizado permite vislumbrar la asimetría de los ecosistemas de programación sometidos a estrés combinatorio:

- **Python:** Es el más afectado por la latencia estructural (tipado dinámico y GIL). En su recorrido base ($n = 30$), sufre una penalización temporal severa en comparación a los otros lenguajes. No obstante, al delegar la complejidad a la heurística inteligente (B&B), su maleabilidad permite resolver instancias de $n = 100$ en menos de un milisegundo (0.000857s), ideal para prototipado rápido.
- **Java:** Presenta un rendimiento altamente competitivo. En la prueba ($n = 30$), la optimización Just-in-Time (JIT) de la máquina virtual procesó más de 1.200 millones de nodos en apenas 3.38 segundos. A pesar de que presenta ligeras oscilaciones por los barridos del Recolector de Basura (Garbage Collector), mantiene una velocidad cercana al çodigo nativo.
- **C++:** Establece el estándar insuperable de optimización matemática. Su acceso restrictivo a punteros, el manejo manual de la pila y el aplanamiento de iteraciones por parte de compilador permitieron que su enfoque Branch and Bound resolviera la instancia máxima de $n = 1000$ en el menor tiempo global (0.003313 segundos).

6. CONCLUSIÓN

El estudio del problema de Programación Entera 0-1 (0-1 IP) nos ha permitido comprender su abstracción matemática pura, y también su comportamiento real en arquitecturas de hardware modernas. Se pudo observar que la naturaleza discreta de sus variables convierte este problema en una herramienta vital para formular decisiones dicotómicas complejas en la industria y en la logística.

También se concluye que la capacidad expresiva del problema también conlleva un costo computacional inherente. Al demostrar teóricamente su NP-completitud, se evidenció que el problema es prácticamente intratable en el peor de los casos, pues la implementación de

Backtracking para la resolución resultó en un colapso brutal frente a la explosión combinatoria.

Sin embargo, la implementación del paradigma heurístico de Branch and Bound nos deja una gran lección: frente a los límites de la clase NP, la solución no reside principalmente en la fuerza bruta, sino en la inteligencia algorítmica. La capacidad de proyectar cotas matemáticas y podar espacios de búsqueda inútiles es lo que permite resolver de forma eficiente instancias de 0-1 IP en tiempos viables.